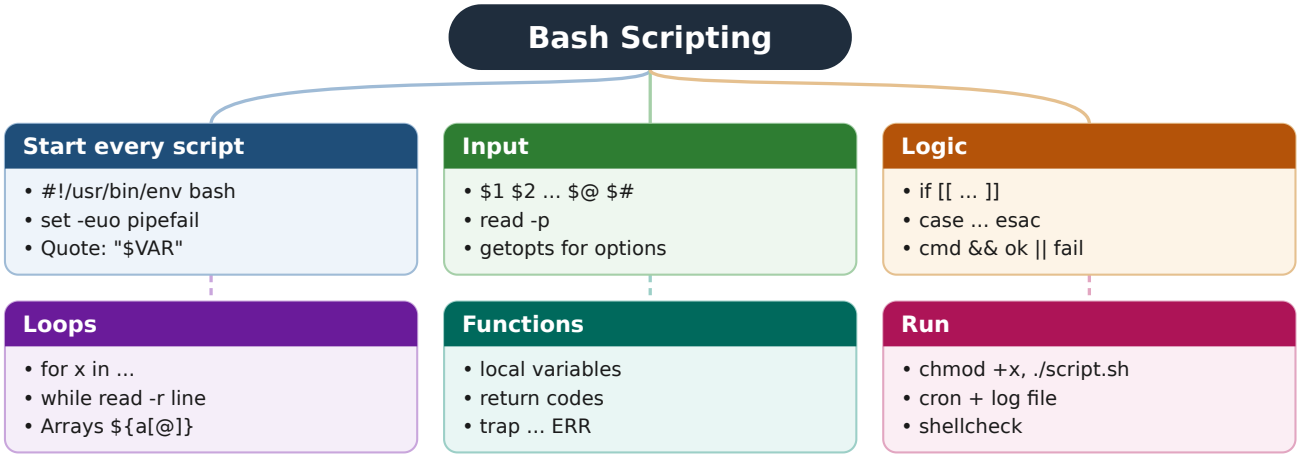
5. Bash Scripting

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 5 of 18 · Thirupathi Keloth · v2.0

Contents

5.0 Why Bash scripting matters
5.1 Scripts, shebang and execution
5.2 Variables and parameter expansion
5.3 Quoting, word splitting and globbing
5.4 Input: arguments, read and options
5.5 Arithmetic
5.6 Conditions
5.7 Loops
5.8 Arrays
5.9 Functions
5.10 Exit codes and error handling
5.11 Output, logging and here-documents
5.12 Working with text, JSON and APIs in scripts
5.13 Debugging
5.14 Production script template
5.15 Real-world scripts
5.16 Common mistakes
5.17 Interview questions and answers
5.18 Self-check: I can...



5.0 Why Bash scripting matters

Bash glues the DevOps world together: CI pipeline steps, Docker entrypoints, server bootstrap (EC2 user data), cron jobs, quick automation around `aws`, `kubectl`, `git` and `curl`. You'll read Bash every day and write it often. For larger programs (complex logic, APIs, data processing), switch to Python.

5.1 Scripts, shebang and execution

```
#!/usr/bin/env bash
# deploy.sh - short description of what the script does
echo "Hello from $(hostname)"
```

Topic	Detail
Shebang `#!`	First line: which interpreter runs the file. `#!/usr/bin/env bash` finds bash in `PATH` (portable); `#!/bin/bash` is also common

Topic	Detail
Make executable	`chmod +x deploy.sh`, then `./deploy.sh`
Run without +x	`bash deploy.sh`
`source deploy.sh` / `. deploy.sh`	Runs in the **current** shell (variables and `cd` persist): used for env files, not for normal scripts
`sh` vs `bash`	`sh` may be dash/POSIX (no arrays, no `[[ ]]`). If you use Bash features, run with bash
Line endings	Scripts edited on Windows may contain `\r` → `bad interpreter: /bin/bash^M`. Fix: `sed -i 's/\r$//' script.sh`

5.2 Variables and parameter expansion

```
name="web01"                # no spaces around =
count=3
readonly APP_DIR="/opt/app"  # constant
echo "$name"                # always quote when using
echo "${name}-backup"       # braces separate the name from following text
unset name
```

Expansion	Result	Example (`file=/var/log/app.log.gz`, `v` unset)
`${v:-default}`	Value, or default if unset/empty	`${ENV:-dev}` → `dev`
`${v:=default}`	Same, and assigns it	
`${v:?message}`	Exit with error if unset/empty	`${DB_HOST:?DB_HOST is required}`
`${v:+alt}`	`alt` if set, else empty	`${DEBUG:+-x}`
`${#file}`	Length	`19`
`${file:0:4}`	Substring (offset, length)	`/var`
`${file#*/}`	Remove shortest prefix match	`var/log/app.log.gz`
`${file##*/}`	Remove longest prefix match → basename	`app.log.gz`
`${file%.*}`	Remove shortest suffix match	`/var/log/app.log`
`${file%%.*}`	Remove longest suffix match	`/var/log/app`
`${file%/*}`	Directory part	`/var/log`
`${file/log/LOG}` / `${file//log/LOG}`	Replace first / all	`/var/LOG/app.log.gz` / `/var/LOG/app.LOG.gz`
`${name^^}` / `${name,,}`	Uppercase / lowercase	`WEB01`

Environment vs shell variables: `export VAR=value` makes it visible to child processes (commands you start). Scripts receive the environment of their caller; configuration for scripts usually comes from environment variables.

5.3 Quoting, word splitting and globbing

The shell processes a command line in order: expansions (`$var` , `$(cmd)` , `$((...))`) → **word splitting** on spaces → **glob** expansion (`*`). Unquoted variables get split and globbed: the source of many bugs.

```
file="my report.txt"
rm $file        # WRONG: tries to delete "my" and "report.txt"
rm "$file"      # RIGHT
```

Form	Use
`"double quotes"`	Default for variables and command substitutions: `"$var"` , `"$(date)"`
`'single quotes'`	Literal text (`'$HOME'` stays `$HOME`), e.g. awk/jq programs
`"$@"`	All arguments, each kept as a separate word: **always use this to pass arguments on**
`$(command)`	Command substitution (prefer over backticks)
`$'...'`	ANSI-C strings: `$'line1\nline2'`

5.4 Input: arguments, read and options

Variable	Meaning
`$0`	Script name
`$1` … `$9` , `${10}`	Positional arguments
`$#`	Number of arguments
`"$@"`	All arguments as separate words
`"$*"`	All arguments as one string
`$?`	Exit code of the last command
`$$`	PID of the script
`$!`	PID of the last background job
`shift`	Drop `$1` , move others down

```
read -r -p "Environment (dev/prod): " env       # -r keeps backslashes literal
read -r -s -p "Password: " pass; echo           # -s hides input (prefer env vars/secret stores)
read -r -t 10 -p "Continue? [y/N] " ans || ans="n"   # -t timeout
```

Options with getopts:

```
usage() { echo "Usage: $0 -e <env> [-n] [-v]" >&2; exit 2; }

env="" dry_run=false verbose=false
while getopts ":e:nvh" opt; do
  case "$opt" in
    e) env="$OPTARG" ;;
    n) dry_run=true ;;
    v) verbose=true ;;
    h) usage ;;
    \?) echo "Unknown option -$OPTARG" >&2; usage ;;
    :)  echo "Option -$OPTARG needs a value" >&2; usage ;;
  esac
done
shift $((OPTIND - 1))                # remaining positional arguments in "$@"
[[ -n "$env" ]] || usage
echo "env=$env dry_run=$dry_run verbose=$verbose rest=$*"
```

5.5 Arithmetic

```
a=7; b=2
echo $(( a + b )) $(( a - b )) $(( a * b ))     # 9 5 14
echo $(( a / b )) $(( a % b )) $(( a ** b ))     # 3 1 49
(( count++ ))                          # increment
(( a > b )) && echo "a is bigger"      # arithmetic test
echo "scale=2; 7/2" | bc               # decimals need bc/awk → 3.50
awk 'BEGIN { printf "%.1f\n", 7/2 }'  # 3.5
```

Bash arithmetic is **integer only**.

5.6 Conditions

```
if [[ -f "$config" && -r "$config" ]]; then
  echo "config readable"
elif [[ "$env" == "prod" ]]; then
  echo "production without config!" >&2
  exit 1
else
  echo "using defaults"
fi
```

Test	True when
`-e path` / `-f file` / `-d dir` / `-L link`	Exists / regular file / directory / symlink
`-r` / `-w` / `-x`	Readable / writable / executable
`-s file`	Exists and not empty
`file1 -nt file2`	file1 newer than file2
`-z "$s"` / `-n "$s"`	String empty / not empty
`"$a" == "$b"` / `!=`	String equal / not equal
`"$s" == web*`	Pattern match (inside `[[ ]]`, unquoted pattern)
`"$s" =~ ^[0-9]+$`	Regex match (`BASH_REMATCH` holds groups)
`$a -eq $b` · `-ne` `-lt` `-le` `-gt` `-ge`	Integer comparisons (or use `(( a < b ))`)

Test	True when
`!` · `&&` · `||`	Not / and / or

`[[ ... ]]` (Bash) is safer than `[ ... ]` (POSIX `test`): no word splitting, supports `&&`, patterns and regex.

A command is a condition: `if grep -q "ERROR" app.log; then ...` (uses the exit code; `-q` = quiet).

case

```
case "$1" in
  start)          systemctl start app ;;
  stop)           systemctl stop app ;;
  restart|reload) systemctl restart app ;;
  *.tar.gz)       tar -xzf "$1" ;;
  *)              echo "Usage: $0 {start|stop|restart}" >&2; exit 2 ;;
esac
```

5.7 Loops

```
for host in web1 web2 db1; do
  ping -c1 -W1 "$host" &>/dev/null && echo "$host up" || echo "$host DOWN"
done

for i in {1..5}; do echo "attempt $i"; done
for (( i=0; i<3; i++ )); do echo "$i"; done

for f in /var/log/*.log; do              # globs are safe with spaces when you quote "$f"
  [[ -e "$f" ]] || continue              # no match → the pattern itself would be returned
  echo "$(wc -l < "$f") lines in $f"
done

while IFS= read -r line; do              # read a file line by line, safely
  echo "user: $line"
done < users.txt

while IFS=, read -r name email team; do  # CSV fields
  echo "$name <$email> -> $team"
done < <(tail -n +2 users.csv)           # skip the header line

until curl -fs http://localhost:8000/health >/dev/null; do   # wait until healthy
  echo "waiting..."; sleep 2
done
```

`break` exits a loop; `continue` skips to the next iteration. **Don't** loop over `$(ls)` or `$(cat file)`: names with spaces break; use globs and `while read`.

5.8 Arrays

```
servers=("web1" "web2" "db 1")
servers+=("cache1")                     # append
echo "${servers[0]}"                    # first element
echo "${#servers[@]}"                   # count
for s in "${servers[@]}"; do echo "$s"; done
echo "${servers[@]:1:2}"                # slice

declare -A port=( [web]=80 [db]=5432 )  # associative array (Bash 4+)
port[redis]=6379
for svc in "${!port[@]}"; do echo "$svc -> ${port[$svc]}"; done

mapfile -t lines < hosts.txt            # read a file into an array
```

5.9 Functions

```
log() {
  local level="$1"; shift
  printf '%s [%s] %s\n' "$(date '+%F %T')" "$level" "$*" >&2
}

is_port_open() {                        # returns status: 0 = yes, 1 = no
  local host="$1" port="$2"
  timeout 3 bash -c ">/dev/tcp/$host/$port" 2>/dev/null
}

get_free_disk_pct() {                   # "returns" data by printing it
  df --output=pcent / | tail -1 | tr -dc '0-9'
}

if is_port_open db.internal 5432; then log INFO "db reachable"; fi
used=$(get_free_disk_pct)
log INFO "root disk used: ${used}%"
```

- `local` keeps variables inside the function (otherwise variables are global).
- `return N` sets a status code (0–255); to return data, **print it** and capture with `$(...)`.
- Functions receive their own `$1`, `$@`, `$#`.

5.10 Exit codes and error handling

Strict mode

```
set -euo pipefail
IFS=$'\n\t'
```

Option	Effect	Watch out
`-e`	Exit when a command fails	Doesn't trigger inside `if`, `&&`, `||` conditions; use `cmd || true` when failure is acceptable
`-u`	Error on unset variables	Use `${VAR:-}` for optional variables
`-o pipefail`	A pipeline fails if any part fails	`grep` with no match returns 1: handle it
`IFS=$'\n\t'`	Split only on newlines/tabs	Safer loops over command output

trap

```
tmpdir=$(mktemp -d)                                    # safe temporary directory
cleanup() { rm -rf "$tmpdir"; }
trap cleanup EXIT                                       # always runs at exit (success or failure)
trap 'echo "Error on line $LINENO (exit $?)" >&2' ERR  # report failures
trap 'echo "Interrupted"; exit 130' INT TERM           # Ctrl+C / kill
```

Exit code conventions

`0` success · `1` general error · `2` misuse/usage error · `126` not executable · `127` command not found · `128+N` killed by signal N (130 = Ctrl+C, 137 = SIGKILL/OOM, 143 = SIGTERM). Always `exit` with a meaningful code so CI/cron/systemd can detect failures.

5.11 Output, logging and here-documents

```
echo "message"                                 # simple output
printf '%-10s %5d %s\n' "web1" 80 "ok"      # formatted output (preferred for exact formats)
echo "error" >&2                               # to stderr
exec > >(tee -a /var/log/myscript.log) 2>&1 # send all further output to screen + log file

cat > /etc/myapp/app.conf <<EOF             # here-document (variables expand)
port=${PORT:-8080}
env=${ENV:-dev}
EOF

cat <<'EOF' > script.txt                    # quoted delimiter: no expansion
literal $HOME and $(date)
EOF
```

Use `logger -t myscript "message"` to write to the system journal/syslog.

5.12 Working with text, JSON and APIs in scripts

```
# Text: grep / awk / sed / cut (see Linux notes)
errors=$(grep -c "ERROR" app.log || true)

# JSON with jq
curl -fsS https://api.github.com/repos/kubernetes/kubernetes | jq -r '.stargazers_count'
aws ec2 describe-instances --output json \
  | jq -r '.Reservations[].Instances[] | [.InstanceId, .State.Name] | @tsv'
kubectl get pods -o json | jq -r '.items[] | select(.status.phase != "Running") | .metadata.name'

# HTTP status checks
code=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 "$url")
[[ "$code" == "200" ]] || { echo "unhealthy: $code" >&2; exit 1; }

# YAML with yq
yq '.spec.replicas' deployment.yaml
```

`curl -f` makes curl return a failure exit code on HTTP errors (≥400); add `--retry 3` for flaky networks.

5.13 Debugging

Tool	Use
`bash -n script.sh`	Syntax check without running

Tool	Use
`bash -x script.sh` / `set -x` … `set +x`	Trace every command as it runs
`PS4='+ ${BASH_SOURCE}:${LINENO}: '`	Show file and line in traces
shellcheck	Static analysis: catches quoting bugs, unused variables, common mistakes (`shellcheck script.sh` ; run it in CI)
`echo "DEBUG: var=$var" >&2`	Quick prints to stderr

5.14 Production script template

```
#!/usr/bin/env bash
# backup-dir.sh - archive a directory with retention
# Usage: backup-dir.sh -s <source_dir> -d <dest_dir> [-k <keep_days>] [-n]
set -euo pipefail

readonly SCRIPT_NAME="${0##*/}"
KEEP_DAYS=7
DRY_RUN=false

log()  { printf '%s [%s] %s\n' "$(date '+%F %T')" "$1" "${*:2}" >&2; }
die()  { log ERROR "$*"; exit 1; }
usage() { echo "Usage: $SCRIPT_NAME -s <src> -d <dest> [-k days] [-n]" >&2; exit 2; }
run()  { if $DRY_RUN; then log DRY "$*"; else "$@"; fi; }

while getopts ":s:d:k:nh" opt; do
  case "$opt" in
    s) SRC="$OPTARG" ;;
    d) DEST="$OPTARG" ;;
    k) KEEP_DAYS="$OPTARG" ;;
    n) DRY_RUN=true ;;
    *) usage ;;
  esac
done
[[ -n "${SRC:-}" && -n "${DEST:-}" ]] || usage
[[ -d "$SRC" ]] || die "source $SRC does not exist"
[[ "$KEEP_DAYS" =~ ^[0-9]+$ ]] || die "keep days must be a number"

exec 9>"/tmp/${SCRIPT_NAME}.lock"           # prevent two runs at the same time
flock -n 9 || die "another instance is running"

run mkdir -p "$DEST"
archive="$DEST/$(basename "$SRC")-$(date +%Y%m%d-%H%M%S).tar.gz"
log INFO "creating $archive"
run tar -czf "$archive" -C "$(dirname "$SRC")" "$(basename "$SRC")"

log INFO "removing archives older than $KEEP_DAYS days"
run find "$DEST" -name "*.tar.gz" -type f -mtime +"$KEEP_DAYS" -delete
log INFO "done"
```

Checklist for real scripts: usage/help · input validation · strict mode · clear logging to stderr · meaningful exit codes · dry-run option for destructive actions · lock to avoid parallel runs · **idempotent** (safe to run twice) · no hard-coded secrets (read from environment/secret manager) · `shellcheck` clean · stored in Git.

5.15 Real-world scripts

Disk usage alert with a webhook notification

```
#!/usr/bin/env bash
set -euo pipefail
THRESHOLD="${THRESHOLD:-80}"
: "${ALERT_WEBHOOK_URL:?set ALERT_WEBHOOK_URL in the environment}"   # secret from env, not code

df --output=pcent,target -x tmpfs -x devtmpfs | tail -n +2 | while read -r pct mount; do
  used="${pct%\%}"
  if (( used >= THRESHOLD )); then
    msg="$(hostname): $mount is ${used}% full"
    curl -fsS -X POST -H 'Content-Type: application/json' \
      -d "{\"text\": \"$msg\"}" "$ALERT_WEBHOOK_URL" >/dev/null
    echo "ALERT: $msg"
  fi
done
```

Service health check with automatic restart

```
#!/usr/bin/env bash
set -euo pipefail
SERVICE="${1:?usage: $0 <service> <url>}"
URL="${2:?usage: $0 <service> <url>}"

if ! curl -fs --max-time 5 "$URL" >/dev/null; then
  logger -t healthcheck "$SERVICE unhealthy, restarting"
  systemctl restart "$SERVICE"
  sleep 5
  if ! curl -fs --max-time 5 "$URL" >/dev/null; then
    logger -t healthcheck "$SERVICE still down"; exit 1
  fi
fi
```

Bulk user creation from CSV

```
#!/usr/bin/env bash
set -euo pipefail
# users.csv: username,group
while IFS=, read -r user group; do
  [[ -z "$user" || "$user" == "username" ]] && continue
  getent group "$group" >/dev/null || sudo groupadd "$group"
  if id "$user" &>/dev/null; then
    echo "exists: $user"
  else
    sudo useradd -m -s /bin/bash -G "$group" "$user"
    echo "created: $user ($group)"
  fi
done < users.csv
```

Wait for a dependency (container entrypoints, CI)

```
#!/usr/bin/env bash
set -euo pipefail
host="${1:?host}" port="${2:?port}" timeout="${3:-60}"
for (( i=0; i<timeout; i++ )); do
  if (exec 3<>"/dev/tcp/$host/$port") 2>/dev/null; then echo "$host:$port is up"; exit 0; fi
  sleep 1
done
echo "timeout waiting for $host:$port" >&2; exit 1
```

AWS automation: list running instances without an Owner tag

```
#!/usr/bin/env bash
set -euo pipefail
REGION="${AWS_REGION:-ap-south-1}"
aws ec2 describe-instances --region "$REGION" \
  --filters Name=instance-state-name,Values=running \
  --query 'Reservations[].Instances[?!not_null(Tags[?Key==`Owner`].Value|[0])].InstanceId' \
  --output text | tr '\t' '\n' | sed '/^$/d'
```

Simple deployment with rollback

```
#!/usr/bin/env bash
set -euo pipefail
APP_DIR=/opt/shop
NEW_VERSION="${1:?usage: $0 <git-tag>}"
previous=$(git -C "$APP_DIR" describe --tags --abbrev=0)

rollback() {
  echo "Deploy failed, rolling back to $previous" >&2
  git -C "$APP_DIR" checkout -q "$previous"
  sudo systemctl restart shop
}
trap rollback ERR

git -C "$APP_DIR" fetch --tags -q
git -C "$APP_DIR" checkout -q "$NEW_VERSION"
"$APP_DIR/.venv/bin/pip" install -q -r "$APP_DIR/requirements.txt"
sudo systemctl restart shop
sleep 3
curl -fs --max-time 5 http://localhost:8000/health >/dev/null
trap - ERR
echo "Deployed $NEW_VERSION"
```

(In real teams, deployments go through CI/CD pipelines and immutable artifacts; scripts like this are for simple servers and learning.)

5.16 Common mistakes

Mistake	Fix
`var = value` (spaces)	`var=value`
Unquoted `$var` with spaces/globs	`"$var"`
`for f in $(ls)`	`for f in ./*`
`cat file | while read line` loses variables (subshell)	`while read -r line; do ...; done < file`
Missing `-r` in `read`	`read -r`
`[ $a == $b ]` with empty variables	`[[ "$a" == "$b" ]]`
Using `==` for numbers	`-eq` or `(( a == b ))`
Ignoring failures	`set -euo pipefail`, check exit codes
Hard-coded passwords/tokens	Environment variables / secret managers
No logs from cron jobs	Redirect output `>> file 2>&1`
CRLF line endings	`dos2unix`, `.gitattributes`

Mistake	Fix
Relying on the current directory	`cd "$(dirname "$0")"` or absolute paths

5.17 Interview questions and answers

#	Question	Answer
1	What is a shebang?	First line `#!` telling the OS which interpreter runs the script
2	`source script.sh` vs `./script.sh`?	Source runs in the current shell (changes persist); `./` runs in a new process
3	`$@` vs `$*`?	`"$@"` keeps each argument separate; `"$*"` joins them into one string
4	What does `set -euo pipefail` do?	Exit on errors, on unset variables, and on failures inside pipelines
5	How do you check the exit status of a command?	`$?` or use the command directly in `if` / `&&` / `||`
6	`[ ]` vs `[[ ]]`?	`[[ ]]` is Bash: safer (no word splitting), supports patterns/regex/ `&&`
7	How do you read a file line by line?	`while IFS= read -r line; do ...; done < file`
8	How do you pass and parse options?	Positional args or `getopts`
9	How do you return a value from a function?	Print it and capture with `$(...)`; `return` only sets the status code
10	What is `trap` used for?	Run cleanup or handlers on EXIT, ERR, INT, TERM
11	How do you debug a script?	`bash -x`, `set -x`, `bash -n`, shellcheck
12	How do you make a script idempotent?	Check state before changing it (`id user`, `mkdir -p`, `grep -q` before appending)
13	How do you avoid hard-coding secrets?	Read them from environment variables or a secrets manager at runtime
14	Exit code 127 / 137?	Command not found / killed by SIGKILL (often OOM)
15	How do you prevent two copies of a cron script running?	`flock` lock file
16	How do you process JSON in Bash?	`jq`
17	Why quote variables?	To prevent word splitting and glob expansion
18	When would you use Python instead of Bash?	Complex logic, data structures, APIs/JSON heavy work, testing, cross-platform tools

5.18 Self-check: I can…

- [ ] Write, make executable and run scripts; explain shebang and `source`.
- [ ] Use variables and parameter expansion (defaults, required, trimming, replacing).
- [ ] Quote correctly and explain word splitting and globbing.
- [ ] Handle arguments, `read`, and options with `getopts`.
- [ ] Do integer arithmetic and know when to use `bc` / `awk`.
- [ ] Write conditions with `[[ ]]`, file/string/number tests, regex and `case`.

- [ ] Write `for`, `while read`, `until` loops safely.
- [ ] Use indexed and associative arrays.
- [ ] Write functions with local variables and proper return codes.
- [ ] Use strict mode, `trap`, `mktemp` and meaningful exit codes.
- [ ] Log properly, use here-documents, process JSON with `jq` and call APIs with `curl`.
- [ ] Debug with `bash -x` and keep scripts shellcheck-clean.
- [ ] Build production-quality scripts (validation, dry-run, locking, idempotency, no secrets).
- [ ] Automate real tasks: alerts, health checks, user management, waiting for services, AWS/kubectl automation.

REMEMBER

- Exit code **0 = success**, anything else = failure (`$?`).
- Tests: `-f` file · `-d` directory · `-z` empty · `-eq/-gt/-lt` numbers · `==` strings.
- Never put passwords in scripts: read them from the environment.